

Multi-Agent Market Simulation & Game-Theoretic Analysis — Complete Polaris OJT Project Documentation

Student(s): Padmanava Parui (251810700240), Nafis Farhan (251810700209) **Year & Section:** 2nd Year, SEM 3A **Track:** Machine Learning / Game Theory / Quantitative Economics **Duration:** 12 Weeks **Team:** 2 Students **Level:** Intermediate–Advanced **Stack:** Python, Ray (RLLib) / PettingZoo, NumPy, Pandas, Nashpy, Streamlit, Plotly (optional C++ core for high-frequency execution) **Dataset:** Simulated environment generation + historical Limit Order Book data (e.g., LOBSTER) for baseline calibration **Constraint:** CPU-first design; GPU used opportunistically for RL training acceleration

Project Description

Designing and implementing a Multi-Agent Market Simulation platform in which independent, self-interested reinforcement-learning (RL) agents trade against one another inside a simulated limit-order-book (LOB) exchange. The platform lets us observe, mathematically and empirically, how game-theoretic strategies emerge from agent interaction, how those strategies affect price discovery and market stability, and where the resulting multi-agent system settles relative to a Nash equilibrium. The project applies software engineering, applied ML/RL, game theory, and technical-documentation best practices across the full 12-week OJT cycle.

Table of Contents

1. Project Description & Overview
2. Business/Research Requirements Document (BRD)
3. Product Requirements Document (PRD)
4. UX Requirements (Dashboard)
5. Technical Requirements Document (TRD)
6. High-Level Design (HLD)
7. Data Design
8. API / Interface Specification
9. Low-Level Design (LLD)
10. Reinforcement-Learning & Game-Theoretic Architecture
11. Security & Simulation-Integrity Design
12. Testing Strategy
13. CI/CD
14. Observability
15. Deployment Architecture
16. Cost Analysis (Compute)
17. 12-Week Project Roadmap
18. Team Responsibilities
19. Repository Structure

20. README (Summary)
 21. Architecture Decision Records
 22. Traceability Matrix
 23. Interview Preparation
 24. Viva / Project Defense
 25. Final Project Quality Score
-

Project Overview

Project Identity

- **Track:** Machine Learning / Game Theory / Quantitative Economics
- **Duration:** 12 weeks
- **Team:** 2 students (Padmanava Parui, Nafis Farhan)
- **Skill level:** Intermediate–Advanced
- **Primary users:** Quantitative research mentors, RL/ML evaluators, the students themselves (as researchers)
- **Constraint:** No dedicated GPU cluster guaranteed; core simulation must remain runnable on CPU, with GPU used only to accelerate RLlib training when available

Executive Summary

Traditional backtesting treats the market as a static time series and assumes a trader's own actions never move the market. Real markets are complex adaptive systems: every order changes the book that the next agent reacts to. This project builds a simulated exchange — a central limit order book (CLOB) — populated by multiple independent RL agents (baseline zero-intelligence traders, market makers, and directional RL traders) who submit and cancel orders every step. Agents update their policies from profit/loss (PnL) rewards through continuous interaction, and a separate game-theoretic analysis layer studies the resulting payoff structure for best-response dynamics and Nash-equilibrium convergence.

The platform is intentionally built as a **research sandbox, not a live-trading system**. No real capital, real exchange connectivity, or real order routing is ever involved.

Core Simulation Flow

Environment initialization → broadcast order-book state to all agents → agents select actions (submit/cancel/modify orders) → matching engine processes the limit order book → rewards (PnL) calculated per agent → agent policies/Q-values updated → metrics logged → game-theoretic analytics computed → Streamlit dashboard updated.

MVP

1. Priority-queue-based Central Limit Order Book (CLOB) with price-time priority matching.
2. Custom PettingZoo/Gym multi-agent environment wrapping the CLOB.

3. Zero-Intelligence baseline agents (random, rule-bound order flow).
4. Market-maker archetype agent (quotes both sides of the book).
5. RL-based directional trader agents (PPO/Q-learning via Ray RLlib).
6. Reward/PnL computation and per-agent inventory tracking.
7. Post-hoc game-theoretic analytics (best-response tracking, Nash-equilibrium estimation via Nashpy).
8. Metrics logging pipeline (episode rewards, convergence curves, book depth, spread).
9. Streamlit + Plotly dashboard for simulation visualization (fallback: static Plotly report).
10. Reproducible, seeded simulation runs.

Out of Scope for MVP

- Real exchange connectivity or live order routing.
 - Real capital or real financial risk of any kind.
 - High-frequency (sub-millisecond) execution guarantees.
 - Real-time Nash-equilibrium computation during training (analysis may run post-hoc on logged data).
 - Production-grade regulatory/compliance modeling.
 - The C++ engine port, adversarial macro shocks, and Shapley-value attribution (all explicit stretch goals — see §2.2 below).
-

Business/Research Requirements Document (BRD)

1. Executive Summary

Quantitative researchers and algorithmic-trading teams need a safe environment to study how independent, learning, self-interested agents interact in a market before ever touching real capital. This project delivers a simulated multi-agent exchange that produces emergent price/volume data purely from agent interaction, plus a game-theoretic analysis layer that explains *why* the market behaves the way it does.

2. Problem Statement

Financial markets are not static time series; they are adversarial, adaptive systems. Backtesting a strategy against historical data silently assumes the strategy has no market impact — which is false for any strategy large enough to matter. This project removes that assumption by making every “trader” in the simulation an autonomous learning agent whose actions genuinely move the book.

3. Vision

Build a reproducible multi-agent RL market simulator that lets researchers safely observe emergent microstructure phenomena (price discovery, volatility clustering, flash-crash-like instability) and quantify the equilibrium behavior of the interacting agents using formal game theory.

4. Objectives

- Build a correct, efficient central limit order book with price-time priority.
- Provide a standard multi-agent RL interface (PettingZoo/Gym) around that book.
- Implement baseline non-learning agent archetypes to validate the environment before any RL is trusted.
- Train RL-based directional trader agents (PPO / Q-learning) against those baselines and against each other.
- Quantify emergent strategic behavior using game-theoretic tools (best-response dynamics, Nash-equilibrium estimation).
- Visualize simulation state and agent performance through an interactive dashboard.
- Keep every run reproducible via fixed random seeds.

5. Personas

Persona	Goals	Pain Points
Quant Research Mentor	Evaluate whether the simulation produces realistic, trustworthy market dynamics	Hard to tell “real” emergent behavior from a buggy engine
RL/ML Engineer (the students)	Train agents that learn stable, improving policies	Multi-agent non-stationarity makes convergence hard to diagnose
Evaluator / Viva Panel	Assess correctness, depth, and defensibility of the work	Needs clear proof the engine is correct and the agents actually learned something

6. Business/Research Use Cases

- Initialize a simulated exchange with configurable agent populations.
- Submit, cancel, and match limit/market orders through a correct CLOB.
- Run RL training loops where agents learn from PnL-based rewards.
- Log per-episode metrics (PnL, Sharpe ratio, inventory, book depth, spread).
- Compute post-hoc game-theoretic metrics (best response, Nash-equilibrium convergence).
- Visualize simulated price/volume time series and agent performance in a dashboard.
- Replay a fixed-seed run for reproducibility during evaluation.

7. Requirements

ID	Requirement	Priority	Acceptance Criteria
BR-001	System can initialize a CLOB with configurable tick size and fee schedule	Must	Book instantiates with valid empty bid/ask queues
BR-002	System matches orders using strict	Must	Matching engine never crosses the

ID	Requirement	Priority	Acceptance Criteria
	price-time priority		spread; FIFO within a price level
BR-003	System exposes a PettingZoo/Gym-compatible multi-agent environment	Must	<code>env.step()</code> / <code>env.reset()</code> conform to the API; passes PettingZoo API test suite
BR-004	System supports Zero-Intelligence and Market-Maker baseline agents	Must	Baselines run end-to-end without RL and produce stable order flow
BR-005	System trains RL directional agents via Ray RLlib	Must	Training loop runs multiple episodes and logs reward curves
BR-006	System computes PnL, Sharpe ratio and inventory per agent	Must	Metrics reproducible from logged order/trade history
BR-007	System estimates Nash equilibria from logged payoff data	Should	Nashpy-based solver returns equilibrium candidates for a given payoff matrix
BR-008	System provides an interactive Streamlit dashboard	Should	Dashboard renders live/replayed price, volume, and agent metrics
BR-009	System supports fixed-seed reproducible runs	Must	Same seed + config reproduces identical trade sequence
BR-010	System validates zero-sum inventory conservation each step	Must	Total fiat + asset inventory across agents is conserved minus fees

8. Non-Functional Requirements

- **Correctness:** the matching engine must never cross the spread and must conserve inventory (minus fees) at every step.
- **Performance:** LOB insertion targeted at $O(\log n)$, best-price retrieval at $O(1)$, so training throughput is not bottlenecked by the engine.
- **Reproducibility:** every run must be replayable from a fixed random seed.
- **Extensibility:** new agent archetypes must be addable without changing the matching engine.

- **Observability:** every episode must emit reward, PnL, and book-state metrics suitable for convergence analysis.
- **Portability:** the core simulation must run on CPU only; GPU is an optional accelerator for RLlib training, not a hard dependency.

9. Success Metrics

- Matching-engine correctness (zero spread-crossing incidents across the full test suite).
- Zero-sum inventory conservation across all logged episodes.
- Convergence of RL agents' reward/loss curves relative to random-action baselines.
- Presence of “stylized facts” of real markets in simulated output (volatility clustering, fat-tailed returns).
- Number/quality of identified Nash-equilibrium candidates from the game-theoretic layer.
- Dashboard usability during live demo/viva.

10. Risks

Risk	Probability	Impact	Mitigation
MARL non-stationarity prevents convergence	High	High	Curriculum: baselines first, freeze-and-train, staged agent introduction
Matching engine has subtle correctness bugs	Medium	High	Zero-sum validation + spread-crossing sanity tests every step
RLlib/PettingZoo integration friction	Medium	Medium	Early spike in Weeks 1–4 before committing to full agent logic
Nash-equilibrium solvers don't converge on real payoff data	Medium	Medium	Fall back to approximate/empirical best-response tracking
Streamlit dashboard slips due to time pressure	Medium	Low	Static Plotly report is an explicitly accepted fallback
C++ port / stretch goals consume core-feature time	Medium	Medium	Stretch goals are explicitly deprioritized (Could) behind all Must items

11. MVP Scope

In scope: CLOB engine, PettingZoo environment, Zero-Intelligence + Market-Maker baselines, RL directional agents, PnL/Sharpe/inventory metrics, post-hoc game-theory analytics, Streamlit/Plotly dashboard, reproducibility, testing.

Out of scope: live trading, real capital, C++ port, adversarial shocks, Shapley attribution (all stretch goals).

12. Future Scope

- C++ core for high-frequency-scale execution speeds.
 - Adversarial macro-economic shock injection and recovery analysis.
 - Shapley-value attribution of which agent behaviors drive volatility.
 - Additional agent archetypes (momentum, mean-reversion, informed traders).
 - Calibration against live/near-real-time LOB feeds beyond the LOBSTER baseline.
-

Product Requirements Document (PRD)

Product Goal

Deliver a correct, reproducible multi-agent market simulator whose emergent price/volume behavior can be explained through formal game-theoretic analysis of the interacting RL agents.

Feature Priorities

Feature	Priority	Notes
Matching Engine (CLOB, price-time priority)	Must	Foundation everything else depends on
Multi-Agent Environment (PettingZoo/Gym)	Must	Required for any RL training to run
Agent Archetypes (Zero-Intelligence, Market Maker)	Must	Needed as a correctness baseline before RL agents mean anything
RL-based directional trader agents	Must	Core research contribution
Game Theory Analytics (best-response, Nash tracking)	Should	Central to the thesis, but ships as post-hoc analysis on logged data rather than real-time
Interactive Streamlit Dashboard	Should	Valuable for demo/viva; a static Plotly report is an acceptable fallback
C++ engine port	Could	Explicit stretch goal

Feature	Priority	Notes
Adversarial shocks	Could	Explicit stretch goal
Shapley value attribution	Could	Explicit stretch goal

User Stories

US-001

As a research mentor, I want to configure agent starting capital, risk tolerance, and allowable action space so that I can control the experiment's initial conditions.

Acceptance criteria

- Agent parameters (capital, risk tolerance, action space: Buy/Sell/Hold/Limit/Market) are configurable at environment init.
- Invalid configurations (e.g., negative capital) are rejected.

US-002

As an RL engineer, I want the environment to expose a standard PettingZoo/Gym API so that I can plug in RLlib training without writing a custom training loop.

Acceptance criteria

- `reset()` and `step()` conform to the PettingZoo Parallel/AEC API.
- Observation and action spaces are explicitly typed.

US-003

As an RL engineer, I want baseline Zero-Intelligence and Market-Maker agents so that I have a correctness and performance floor before trusting any RL result.

US-004

As a researcher, I want per-agent PnL, Sharpe ratio, and inventory logged every episode so that I can evaluate whether learning is actually happening.

US-005

As a researcher, I want post-hoc game-theoretic analysis (best-response dynamics, Nash-equilibrium estimation) on the logged payoff data so that I can explain *why* the market settled where it did.

US-006

As a viewer (mentor/evaluator), I want an interactive dashboard showing simulated price, volume, and agent performance so that I can inspect a run without reading raw logs.

US-007

As an evaluator, I want every run to be reproducible from a fixed seed so that I can independently verify claimed results.

Product States

ENV_INITIALIZED → AGENTS_REGISTERED → EPISODE_RUNNING →
EPISODE_COMPLETE → METRICS_LOGGED → ANALYSIS_READY

Failure state: SIMULATION_INVALID (raised when a zero-sum or spread-crossing check fails).

Product Principles

- Correctness before cleverness — the matching engine must be provably correct before any RL result is trusted.
 - Baselines before RL — Zero-Intelligence and Market-Maker agents validate the environment first.
 - Reproducibility over convenience — every run is seeded and replayable.
 - Post-hoc analysis is acceptable — real-time game theory is a “Should,” not a “Must.”
 - No unsupported claims about “learned” strategy without a convergence curve to back it up.
-

UX Requirements (Dashboard)

Information Architecture

1. Run Configuration (agent counts, capital, risk tolerance, tick size, fees)
2. Live/Replay Simulation View
3. Order Book Depth Chart
4. Price & Volume Time Series
5. Agent Performance (PnL, Sharpe, inventory)
6. Game-Theoretic Analytics (best-response trace, equilibrium candidates)
7. Convergence Diagnostics (reward/loss curves)
8. Run History / Reproducibility (seed, config snapshot)

Primary User Flow

Configure Run → Start Simulation → Watch Order Book & Price Series Update → Inspect Agent Performance → Review Game-Theoretic Analytics → Check Convergence Diagnostics → Save/Export Run.

Key Screen Requirements

Run Configuration

- Agent archetype selection and counts.

- Starting capital / risk tolerance per archetype.
- Tick size, fee schedule, episode length.
- Random seed input (for reproducibility).

Order Book / Price Chart

- Live depth-of-book visualization (bid/ask ladders).
- Simulated price and volume time series (Plotly).
- Spread indicator with a visible “never crosses” sanity marker.

Agent Performance

- Per-agent PnL over time.
- Sharpe ratio and inventory accumulation.
- Baseline-vs-RL comparison view (does RL outperform random?).

Game-Theoretic Analytics

- Best-response trace per agent.
- Nash-equilibrium candidate table (from Nashpy on logged payoff matrices).
- Convergence/stability indicator.

Convergence Diagnostics

- Reward and loss curves per training episode.
- Flag for oscillating vs. converging behavior.

States

Every screen must define loading, empty (no run yet), success, and error (e.g., SIMULATION_INVALID) states.

Accessibility

- Keyboard navigation across dashboard tabs.
 - Semantic labels for all charts and tables.
 - Adequate color contrast; never rely on color alone to distinguish bid vs. ask.
 - Responsive layout for demo/viva projection.
-

Technical Requirements Document (TRD)

Proposed Architecture

- **Simulation core:** Python — Central Limit Order Book, matching engine
- **Multi-agent environment:** PettingZoo (MARL standard) / OpenAI Gym API
- **RL training:** Ray RLlib (PPO, Q-learning agents)
- **Baseline agents:** Zero-Intelligence, Market-Maker (rule-based, no learning)
- **Game theory:** Nashpy (equilibrium solvers, e.g., Lemke-Howson), NumPy/Pandas for payoff-matrix construction

- **Visualization/UI:** Streamlit + Plotly (fallback: static Plotly report)
- **Optional stretch:** C++ core for the matching engine, wrapped into Python via bindings

Technology Rationale

PettingZoo + Gym

Industry-standard MARL interface; lets RLlib and other RL libraries plug in without a bespoke training loop.

Ray RLlib

Mature, scalable multi-agent RL library supporting PPO and other on-policy/off-policy algorithms out of the box, with distributed training support if compute allows.

Priority-Queue LOB (Python, optional C++ later)

Dual priority queues (bids/asks) give $O(\log n)$ insertion and $O(1)$ best-price retrieval — sufficient for research-scale simulation; a C++ port is reserved as a stretch goal only if throughput becomes a bottleneck.

Nashpy

Lightweight, well-tested library for classical game-theoretic equilibrium computation (e.g., Lemke-Howson), avoiding the need to hand-implement equilibrium solvers.

Streamlit + Plotly

Fast to build, good for interactive research dashboards and demo/viva presentation; Plotly's static export is an accepted fallback if the interactive dashboard slips.

Technical Requirements

ID	Requirement	Maps to
TR-001	Priority-queue CLOB with price-time priority	BR-001, BR-002
TR-002	PettingZoo/Gym-compliant multi-agent environment	BR-003
TR-003	Zero-Intelligence & Market-Maker baseline agents	BR-004
TR-004	RLlib-based PPO/Q-learning training loop	BR-005
TR-005	Per-agent PnL/Sharpe/inventory computation	BR-006
TR-006	Nashpy-based post-hoc equilibrium analysis	BR-007
TR-007	Streamlit + Plotly dashboard	BR-008

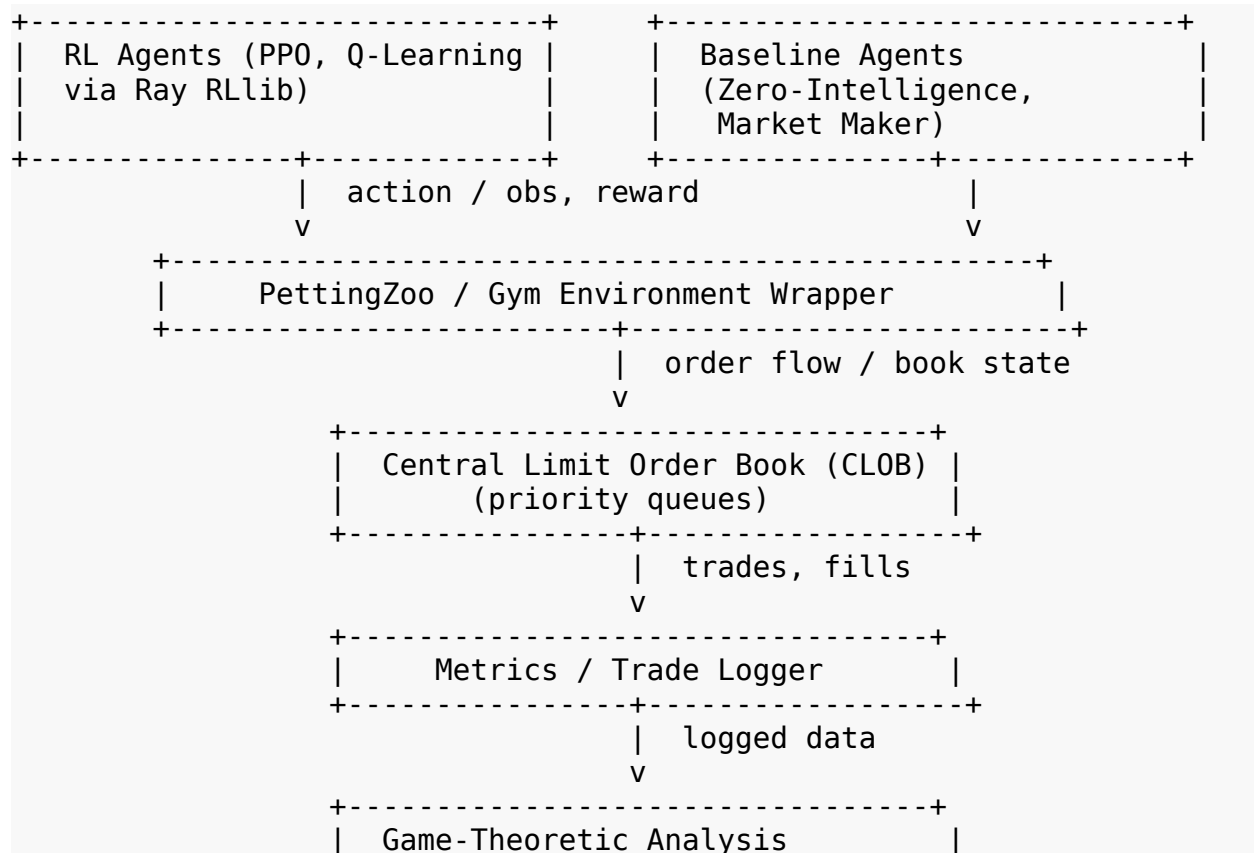
ID	Requirement	Maps to
TR-008	Seeded, reproducible simulation runs	BR-009
TR-009	Zero-sum inventory + spread-crossing validation checks	BR-010
TR-010	Metrics/logging pipeline for all of the above	BR-006, BR-007

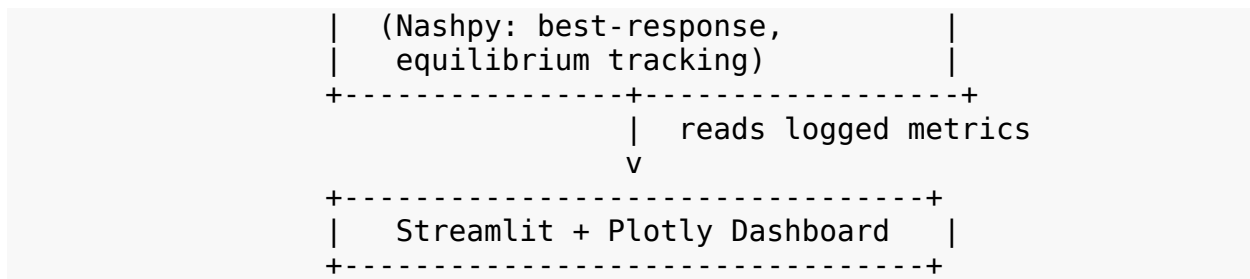
NFR Targets

- Matching-engine insertion: target $O(\log n)$; best-price retrieval: target $O(1)$.
- Zero spread-crossing incidents across the full test suite.
- Inventory conservation error: 0 (minus explicitly modeled fees) at every step.
- Fixed-seed runs must be bit-for-bit reproducible in trade sequence.
- Training loop must run unattended for the full episode budget without manual restarts.

High-Level Design (HLD)

Architecture





Components

Component	Responsibility
Central Limit Order Book (CLOB)	Maintains bid/ask priority queues; matches orders on price-time priority; enforces tick size and fees
PettingZoo/Gym Environment Wrapper	Exposes <code>reset()</code> / <code>step()</code> ; broadcasts book state as observations; routes agent actions into the CLOB
Baseline Agents	Zero-Intelligence (random/rule-bound order flow) and Market-Maker (quotes both sides) — validate the environment before RL is trusted
RL Agents (Ray RLlib)	PPO / Q-learning directional traders; update policy from PnL-based rewards
Metrics / Trade Logger	Records every order, trade, PnL, Sharpe, and inventory value per step/episode
Game-Theoretic Analysis Layer	Builds payoff matrices from logged data; runs Nashpy solvers (e.g., Lemke-Howson) for equilibrium candidates; tracks best-response dynamics
Streamlit + Plotly Dashboard	Visualizes live/replayed price, volume, order-book depth, agent performance, and equilibrium analytics

End-to-End Simulation Flow

1. Environment initializes with configured agent population, capital, and market rules.
2. Environment broadcasts the current order-book state as an observation to all agents.
3. Each agent (baseline or RL) selects an action (submit/cancel/modify an order).
4. The matching engine processes the resulting order flow against the CLOB (price-time priority).
5. Rewards (PnL) are computed per agent from the resulting fills and inventory change.
6. RL agents update their policy/Q-values from the reward signal (Bellman-style update).
7. Metrics (PnL, Sharpe, inventory, book depth, spread) are logged for the step/episode.
8. After training/episodes complete, the game-theoretic analysis layer builds payoff matrices and runs Nashpy equilibrium solvers on the logged data.

9. The Streamlit dashboard reads logged metrics and analytics for live or replayed visualization.

Scalability

Start as a single-process simulation (CLOB + environment + agents in one Python process) for research-scale episode counts. If throughput becomes a bottleneck, introduce the stretch-goal C++ core for the matching engine only, keeping the RL/game-theory layers unchanged (isolated by the PettingZoo interface boundary).

Reliability

- Deterministic, seeded RNG for every run.
- Zero-sum inventory check and spread-crossing check run every step, not just at test time — a violation immediately raises `SIMULATION_INVALID` rather than silently producing bad data.
- Training checkpoints saved periodically so a crashed run can resume rather than restart from scratch.

Security / Integrity

This is a research sandbox with no real capital or external connectivity, so “security” here means **simulation integrity**, not attacker defense:

- No live exchange connectivity — the CLOB is fully self-contained.
 - Config and seed are versioned alongside results so a claimed run can be independently reproduced.
 - No hidden state; every agent’s observation is derived only from the broadcast order-book state (no information leakage between agents beyond what the market itself reveals).
-

Data Design

Because this is a simulation/research platform rather than a transactional application, “data design” here covers the **logged simulation record schema** (written to Parquet/CSV/SQLite rather than a production database).

Core Data Entities

SimulationRun

- `run_id`
- `seed`
- `config_snapshot` (agent counts, capital, tick size, fees)
- `started_at`
- `status` (`RUNNING` / `COMPLETE` / `SIMULATION_INVALID`)

Agent

- agent_id
- run_id
- archetype (ZERO_INTELLIGENCE / MARKET_MAKER / RL_DIRECTIONAL)
- starting_capital
- risk_tolerance
- policy_version (for RL agents)

Order

- order_id
- run_id
- agent_id
- side (BID / ASK)
- order_type (LIMIT / MARKET)
- price
- quantity
- timestamp_step
- status (OPEN / FILLED / PARTIALLY_FILLED / CANCELLED)

Trade

- trade_id
- run_id
- buy_order_id
- sell_order_id
- price
- quantity
- timestamp_step

AgentMetric (per episode)

- run_id
- agent_id
- episode
- pnl
- sharpe_ratio
- inventory
- reward

EquilibriumAnalysis

- run_id
- payoff_matrix_ref
- equilibrium_candidates (JSON-encoded strategy profiles)
- method (LEMKE_HOWSON / other Nashpy method)
- computed_at

Entity Relationships

- One `SimulationRun` has many `Agent`, `Order`, `Trade`, `AgentMetric`, and (typically one) `EquilibriumAnalysis` record.
- Each `Trade` references exactly one buy `Order` and one sell `Order`.
- Each `AgentMetric` row references exactly one `Agent` and the `SimulationRun` it belongs to.

Indexes / Access Patterns

- `Order(run_id, timestamp_step)` — for replay in time order.
- `Trade(run_id, timestamp_step)` — for price/volume time-series reconstruction.
- `AgentMetric(run_id, agent_id, episode)` — for per-agent convergence curves.

Data Lifecycle

Simulate → log orders/trades/metrics → validate (zero-sum + spread checks) → post-hoc game-theoretic analysis → dashboard visualization → archive run (config + seed + logs) for reproducibility.

All data is synthetic/simulated except the optional LOBSTER historical data used purely for baseline calibration of the “background noise” agents — no real customer or account data is ever involved.

API / Interface Specification

Because the “clients” of this system are RL training code and the dashboard (not external HTTP consumers), the primary interface is the **PettingZoo/Gym environment API**, plus a small internal metrics-query interface used by the dashboard.

Environment Interface (PettingZoo / Gym)

`env.reset(seed=None)`

Initializes a new episode with a fresh order book and agent population; returns the first observation set.

`env.step(actions)`

Accepts a dict of `{agent_id: action}` (Buy/Sell/Hold/Limit/Market with price/quantity where applicable); returns (observations, rewards, terminations, truncations, infos).

`env.observation_space(agent_id) / env.action_space(agent_id)`

Explicitly typed spaces describing the visible order-book state and the allowable action set per agent.

Metrics Query Interface (internal, used by dashboard)

`get_run_summary(run_id)`

Returns run config, seed, and status.

`get_price_series(run_id)`

Returns the simulated time-series of trade prices and volumes.

`get_agent_metrics(run_id, agent_id=None)`

Returns per-agent PnL, Sharpe ratio, and inventory history.

`get_equilibrium_analysis(run_id)`

Returns the logged Nashpy equilibrium candidates and best-response trace for the run.

Standard Error Shape

```
{
  "error": {
    "code": "SIMULATION_INVALID",
    "message": "Spread-crossing detected at step 4213.",
    "run_id": "run_2026_09_08_001"
  }
}
```

Use `SIMULATION_INVALID` for correctness-check failures, `CONFIG_ERROR` for invalid run configuration, and `NOT_FOUND` when a queried `run_id`/`agent_id` doesn't exist.

Low-Level Design (LLD)

Module Layout

```
src/
  engine/
    order_book.py          # dual priority-queue bid/ask book
    matching_engine.py     # price-time priority matching
    validators.py          # zero-sum + spread-crossing checks
  env/
    market_env.py          # PettingZoo/Gym wrapper around the engine
    spaces.py              # observation/action space definitions
  agents/
    zero_intelligence.py
    market_maker.py
    rl_directional.py      # RLlib PP0/Q-learning policy wrapper
  gametheory/
    payoff_builder.py      # builds payoff matrices from logged data
    nash_solver.py         # Nashpy (Lemke-Howson) wrapper
```

```

    best_response.py
metrics/
  logger.py          # writes Order/Trade/AgentMetric records
  pnl.py             # PnL, Sharpe, inventory calculations
dashboard/
  app.py             # Streamlit entrypoint
  charts.py          # Plotly chart builders
tests/

```

Key Interfaces

OrderBook

- `add_order(order)`
- `cancel_order(order_id)`
- `best_bid() / best_ask()`
- `depth(levels)`

MatchingEngine

- `process(order) -> list[Trade]`
- `validate_no_cross()`
- `validate_zero_sum(agents)`

AgentPolicy (shared interface for baseline + RL agents)

- `act(observation) -> action`
- `update(reward, next_observation)` (no-op for non-learning baselines)

NashSolver

- `build_payoff_matrix(logged_trades, agents)`
- `solve(method="lemke_howson")`
- `best_response_trace(agents)`

Sequence (single simulation step)

1. Env --broadcast(book state)--> Agents
2. Agents --actions (orders)--> Env
3. Env --process(order)--> MatchingEngine
4. MatchingEngine --trades--> Env
5. Env --reward, next observation--> Agents
6. Env --log(orders, trades, metrics)--> Logger

(after the episode / run completes)

7. Logger --logged data--> NashSolver (post-hoc)
8. NashSolver --build_payoff_matrix() then solve()

Design Patterns

- **Strategy pattern** for interchangeable agent archetypes behind the shared AgentPolicy interface.

- **Adapter pattern** to keep RLlib-specific policy code isolated behind `rl_directional.py`, so the environment doesn't depend on RLlib internals.
- **Repository-style logger** (`metrics/logger.py`) as the single write path for Order/Trade/AgentMetric records — no other module writes logs directly.
- Avoid introducing patterns (e.g., microservices) that aren't justified by the single-process, research-scale nature of this project.

Validation

Validate order parameters (positive price/quantity, valid side/type), agent-config bounds (non-negative capital), and — every single step, not just in tests — the zero-sum inventory and no-spread-crossing invariants.

Error Handling

Every validator failure raises a `SimulationInvalidError` carrying the failing step and a stable error code (`SIMULATION_INVALID`, `CONFIG_ERROR`); the simulation halts rather than continuing on corrupted state, since silently continuing would invalidate every downstream metric and equilibrium claim.

Reinforcement-Learning & Game-Theoretic Architecture

Why Multi-Agent RL + Game Theory (and not a single predictive model)?

A single supervised model predicting “the next price” cannot capture the fact that every trader's action changes the market that every other trader reacts to. Multi-agent RL lets each agent learn its own policy under genuinely adversarial, non-stationary conditions, while the game-theoretic layer gives a principled vocabulary (best response, Nash equilibrium) for explaining the resulting collective behavior — rather than just eyeballing price charts.

Recommended Architecture

```

1. Order-Book State --> Agent Observation
2. Agent Observation --> Policy (PPO / Q-learning / rule-based baseline)
3. Policy --> Action (order submitted)
4. Action (order) --> Matching Engine
5. Matching Engine --> Trade / Reward (PnL)
6. Trade / Reward (PnL) --> Policy Update (Bellman-style)
7. Policy Update --> Logged Metrics

```

(after training / episodes complete)

```

8. Logged Metrics --> Payoff Matrix Construction (Nashpy)
9. Payoff Matrix --> Best-Response Tracking + Nash-Equilibrium Solve

```

Core Algorithms

Q-Learning / PPO (agent policy updates)

Driven by the Bellman equation for value/policy updates:

$$Q(s,a) \leftarrow Q(s,a) + \alpha [R + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a)]$$

- s, a : current order-book state and chosen order action.
- R : PnL-based reward from the resulting fill/inventory change.
- γ : discount factor; α : learning rate.

Nash-Equilibrium Solvers (game-theoretic layer)

Algorithms such as **Lemke-Howson** (via Nashpy) analyze the logged payoff matrices to find strategy profiles where no agent can unilaterally improve its reward by deviating — i.e., candidate Nash equilibria for the emergent multi-agent game.

Training Curriculum

1. Validate the matching engine and environment with **only baseline agents** (Zero-Intelligence, Market-Maker) — no RL yet.
2. Introduce a **single RL agent** against the frozen baselines to confirm it can learn to beat random action.
3. Introduce **multiple simultaneously-learning RL agents**, accepting non-stationarity as the central research challenge (see BRD Risks).
4. Run **post-hoc game-theoretic analysis** on the logged multi-agent payoff data once training stabilizes.

Guardrails / Correctness Checks

- Zero-sum validation: total fiat + asset inventory across all agents must balance every step (minus modeled fees).
- Spread-crossing check: best bid must always remain strictly below best ask.
- Convergence tracking: reward/loss curves must be plotted and inspected — oscillation is treated as a red flag, not “still learning.”
- Fixed random seed for full reproducibility of any claimed result.

Evaluation

Measure:

- RL agents' PnL/Sharpe vs. the Zero-Intelligence baseline (behavioral proof of learning).
 - Presence of stylized facts (volatility clustering, fat-tailed returns) in simulated price data.
 - Number and stability of identified Nash-equilibrium candidates.
 - Wall-clock/episode throughput of the matching engine (relevant if the C++ stretch goal is pursued).
-

Security & Simulation-Integrity Design

There is no real capital, real user data, or external network exposure in this project, so this section focuses on **integrity of the research result** rather than an attacker threat model.

Reproducibility Controls

- Every run stores its random seed and full config snapshot alongside results.
- Policy checkpoints are versioned so a claimed learning curve can be re-derived.

Data Integrity

- Zero-sum inventory and no-spread-crossing checks run every step (not just in the test suite) and immediately halt the run on violation.
- Logged Order/Trade/AgentMetric records are append-only within a run.

Environment Isolation

- No live exchange connectivity of any kind — the CLOB is entirely self-contained and simulated.
- Historical LOBSTER data, if used, is read-only calibration input, never a live feed.

Dashboard/Access

- Streamlit dashboard is read-only against logged run data; it never writes back into a live simulation from the UI.
 - No secrets or credentials are required anywhere in this pipeline (no external APIs), so there is no secret-management surface to defend.
-

Testing Strategy

Test Levels

1. Unit (order book, matching engine, PnL/Sharpe calculations)
2. Integration (environment $\leftrightarrow$ matching engine $\leftrightarrow$ agents)
3. Environment-API conformance (PettingZoo/Gym API test suite)
4. RL training (reward/loss convergence checks)
5. Game-theory (Nashpy solver correctness on known toy payoff matrices)
6. Dashboard/UI (Streamlit rendering of logged runs)
7. Reproducibility (seeded-run replay equality)

Sample Test Cases

ID	Scenario	Expected Result	Priority
TEST-001	Submit a valid limit order	Order enters the correct priority queue at the correct price level	High

ID	Scenario	Expected Result	Priority
TEST-002	Two crossing orders submitted	Immediate match at correct price with correct quantity	High
TEST-003	Partial fill on a large market order in low liquidity	Order marked <code>PARTIALLY_FILLED</code> ; remainder stays open or cancels per rules	High
TEST-004	Spread-crossing sanity check after a batch of orders	Best bid always strictly below best ask	High
TEST-005	Zero-sum inventory check after N random steps	Total fiat+asset inventory conserved minus fees	High
TEST-006	RL agent trained vs. frozen Zero-Intelligence baseline	RL agent's PnL exceeds baseline's after training	High
TEST-007	Same seed run twice	Identical trade sequence produced both times	High
TEST-008	Nashpy solver on a known toy 2x2 payoff matrix	Returns the analytically known equilibrium	Medium
TEST-009	PettingZoo API conformance suite	Passes without errors	High
TEST-010	Streamlit dashboard renders a completed run	Price/volume, agent metrics, and equilibrium tables all display	Medium

RL Evaluation

Track reward and loss curves across training episodes for each agent; compare against the frozen-baseline PnL floor; treat oscillating (non-converging) curves as a finding to investigate, not a result to report as “learning.”

Game-Theory Evaluation

Validate the Nashpy solver against known toy games with analytically known equilibria before trusting its output on the logged multi-agent payoff data.

CI/CD

Pipeline

```
Developer
|
Feature Branch
|
Pull Request
|
Lint + Unit Tests (order book, PnL/Sharpe)
|
Integration Tests (env + agents + matching engine)
|
PettingZoo API Conformance Check
|
Reproducibility Check (seeded replay diff)
|
Build
|
Demo/Viva Environment Deployment
```

Branching

- main: stable, demo-ready state
- develop: integration branch for in-progress agent/game-theory work
- feature/*: individual student work (e.g., feature/lob-engine, feature/rl-agents)

Pull Requests

Every PR should contain:

- Problem being solved
- Solution summary
- Tests added/passed
- Screenshots of dashboard changes where applicable
- Risks/known limitations

Quality Gates

Do not merge when the zero-sum/spread-crossing tests fail, when the PettingZoo conformance suite fails, or when a reproducibility check produces a non-identical replay for the same seed.

Observability

Logs

Log per step/episode:

- `run_id, episode, step`
- Agent ID and action taken
- Resulting trades (if any)
- Validation status (OK / SIMULATION_INVALID)

Metrics

- Reward per agent per episode
- PnL, Sharpe ratio, inventory per agent
- Order-book depth and bid-ask spread over time
- Matching-engine throughput (orders/sec) — relevant if the C++ stretch goal is pursued
- Nash-equilibrium candidate count and stability across runs
- Training wall-clock time per episode

Tracing

Trace: Agent action → Environment step → Matching engine → Trade/Reward
→ Logger → (post-hoc) Game-theory analysis → Dashboard.

Alerts (research-context equivalents)

Flag/halt on:

- Any spread-crossing event.
 - Any zero-sum inventory violation.
 - Reward curves diverging (exploding loss) during training.
 - Reproducibility check failing on a re-run with the same seed.
-

Deployment Architecture

Environments

Local development → CI test runs → Demo/Viva environment.

Suggested Deployment

- Core simulation (engine + environment + RLlib training) run as a local/CPU Python process; GPU used opportunistically for RLlib if available.

- Streamlit dashboard deployed locally (or on a lightweight host) reading from the logged run store (Parquet/CSV/SQLite) — no live production infrastructure is required for a research OJT deliverable.
- Optional C++ core (stretch goal), if built, is compiled and wrapped via Python bindings without changing the environment’s external interface.

Compute Constraint Handling

Because a dedicated GPU is not guaranteed:

1. Keep the matching engine and baseline agents CPU-only by default.
2. Use RLlib’s CPU execution path for the MVP; enable GPU only opportunistically for larger training runs.
3. Do not assume GPU availability in the reproducibility pipeline — a CPU re-run must still validate results.
4. Record wall-clock training time per configuration so CPU-vs-GPU differences are documented if both are tested.

Rollback

Keep versioned policy checkpoints and config snapshots so a run can be reverted to a previously validated agent/engine configuration.

Cost Analysis (Compute)

Because this is a research OJT project with no paid API/user-facing costs, “cost” here means **compute cost of running the simulation and training**, not a per-user financial model.

Cost Model

Estimate compute cost from:

- Episodes per training run
- Steps per episode
- Number of concurrent RL agents being trained
- CPU-hours (baseline) vs. optional GPU-hours (RLlib acceleration)
- Storage for logged Order/Trade/AgentMetric records per run

Optimization

- Keep the matching engine efficient (priority-queue design) so CPU time is spent on learning, not on book bookkeeping.
- Batch/vectorize baseline-agent order generation where possible.
- Only escalate to GPU training when CPU throughput is demonstrably the bottleneck.

- Prune verbose per-step logging to what's needed for the zero-sum/spread checks and metrics, avoiding unnecessary storage growth across long training runs.
-

12-Week Project Roadmap

Phase	Weeks	Planned Deliverables	Mentor Checkpoint
Core Engine	1-4	Priority-queue LOB matching engine; custom PettingZoo environment setup	☐
Agent Logic	5-8	Baseline Zero-Intelligence agents and initial RL training loop implementation	☐
Game Theory	9-11	Advanced agent interactions, Nash-equilibrium calculations, and metric logging	☐
Executive	12	Streamlit dashboard deployment and final simulation report	☐

Definition of Done

A feature is complete when it is: implemented, unit- and integration-tested, validated against the zero-sum/spread-crossing checks, logged/observable, documented, and demonstrable in the deployed demo/viva environment.

Team Responsibilities

Padmanava Parui — Agent Policies, RL Logic & Game Theory

- Agent policy design (Zero-Intelligence baseline logic feeding into RL agent design).

- RL training loop implementation via Ray RLlib (PPO/Q-learning).
- Reward/PnL shaping and policy-update logic.
- Game-theoretic analysis: payoff-matrix construction, Nashpy-based equilibrium solving, best-response tracking.

Nafis Farhan — LOB Engine, Environment & Dashboard/Documentation

- Central Limit Order Book implementation (priority-queue bid/ask structures).
- Matching-engine correctness (price-time priority, partial fills, spread-crossing prevention).
- Custom PettingZoo/Gym environment wrapper around the engine.
- Streamlit + Plotly dashboard UI.
- Documentation consolidation and formatting.

Shared

- HLD/LLD architecture (both students must be able to defend the full architecture at viva).
- Zero-sum/spread-crossing validation testing.
- Metrics logging pipeline integration.
- CI/CD, reproducibility checks, and final report/demo preparation.

Both students must understand the complete architecture — engine, environment, agents, and game-theory layer — and be able to defend all of it during evaluation.

Repository Structure

```
multi-agent-market-simulation/
├── docs/
│   ├── 01-project-overview.md
│   ├── 02-brd.md
│   ├── 03-prd.md
│   ├── 04-ux-requirements.md
│   ├── 05-trd.md
│   ├── 06-hld.md
│   ├── 07-data-design.md
│   ├── 08-api-specification.md
│   ├── 09-lld.md
│   ├── 10-rl-gametheory-architecture.md
│   ├── 11-security-integrity.md
│   ├── 12-testing.md
│   ├── 13-cicd.md
│   ├── 14-observability.md
│   ├── 15-deployment.md
│   ├── 16-cost-analysis.md
│   ├── 17-roadmap.md
│   └── 18-team-responsibilities.md
```

```
├── 20-readme.md
├── 21-adrs.md
├── 22-traceability.md
├── 25-project-score.md
├── src/
│   ├── engine/
│   ├── env/
│   ├── agents/
│   ├── gametheory/
│   ├── metrics/
│   └── dashboard/
├── tests/
├── data/          # LOBSTER calibration data (read-only)
├── scripts/
├── .github/workflows/
├── README.md
├── CONTRIBUTING.md
├── LICENSE
└── .gitignore
```

README (Summary)

Problem

Traditional backtesting assumes a trader's actions don't move the market. Real markets are adversarial, adaptive systems.

Solution

A multi-agent market simulator where independent RL agents trade against each other inside a correct central limit order book, with a game-theoretic layer explaining the emergent equilibrium behavior.

Key Features

- Priority-queue central limit order book with price-time priority matching
- PettingZoo/Gym-compliant multi-agent environment
- Zero-Intelligence and Market-Maker baseline agents
- RL-based directional trader agents (Ray RLlib: PPO / Q-learning)
- Per-agent PnL, Sharpe ratio, and inventory metrics
- Post-hoc game-theoretic analysis (best-response, Nash-equilibrium via Nashpy)
- Streamlit + Plotly interactive dashboard
- Fixed-seed reproducibility

Architecture

RL Agents (RLlib) + Baseline Agents – PettingZoo/Gym Environment – Central Limit Order Book – Metrics Logger – Game-Theoretic Analysis (Nashpy) – Streamlit/Plotly Dashboard.

Tech Stack

Python, Ray RLlib, PettingZoo, NumPy, Pandas, Nashpy, Streamlit, Plotly (optional C++ core as a stretch goal).

Important Disclaimer

This is a research/OJT simulation prototype. It involves no real capital, no live exchange connectivity, and must not be interpreted as a live-trading system.

Running

Document Python/library versions and any RLlib/GPU configuration separately; keep the CPU-only path as the default so the project remains runnable without a dedicated GPU.

Architecture Decision Records

ADR-001 — PettingZoo/Gym over a Custom Environment Interface

Context: The environment must support multiple simultaneously-learning agents. **Decision:** Use the PettingZoo standard (Gym-compatible) rather than a bespoke interface. **Rationale:** Plugs directly into Ray RLlib and other MARL tooling without a custom training loop. **Trade-off:** Some constraints imposed by the standard API shape.

ADR-002 — Priority-Queue LOB (Python) with C++ Port as Stretch Goal

Decision: Implement the matching engine in Python using dual priority queues first; only port to C++ if profiling shows it's the bottleneck. **Rationale:** Keeps the MVP achievable within the 12-week window; $O(\log n)$ insertion / $O(1)$ best-price retrieval is sufficient at research scale. **Trade-off:** Lower raw throughput than a native C++ engine, acceptable given the project's research (not HFT-production) scope.

ADR-003 — Ray RLlib for RL Training

Decision: Use Ray RLlib (PPO/Q-learning) rather than a hand-rolled training loop. **Rationale:** Mature, well-tested MARL support; frees time for the game-theory research contribution. **Trade-off:** Additional dependency surface and a learning curve for RLlib's configuration API.

ADR-004 — Game-Theoretic Analysis Runs Post-Hoc, Not Real-Time

Decision: Nash-equilibrium/best-response analysis runs on logged data after training/episodes complete, not inside the live training loop. **Rationale:** Real-time equilibrium computation would slow training and isn't required for the research conclusions; this is explicitly a "Should," not a "Must," per the PRD. **Trade-off:** Equilibrium insights are only available after a run finishes, not live during training.

ADR-005 — Streamlit + Plotly with a Static-Report Fallback

Decision: Build the dashboard in Streamlit/Plotly; accept a static Plotly report as a fallback if time runs short. **Rationale:** Protects the core research deliverables (engine, agents, game theory) from being crowded out by UI polish under the 12-week deadline. **Trade-off:** A static fallback is less demo-friendly than a live interactive dashboard.

Traceability Matrix

Business/ Research	Product Feature	Technical	Component	Data	Test
BR-001, BR-002	Matching Engine	TR-001	engine/ order_book.py, engine/matching_engine.py	Order, Trade	TEST-001-005
BR-003	Multi-Agent Environment	TR-002	env/ market_env.py	SimulationRun	TEST-009
BR-004	Baseline Agents	TR-003	agents/ zero_intelligence.py, agents/market_maker.py	Agent	TEST-001-005
BR-005	RL Directional Agents	TR-004	agents/ rl_directional.py	AgentMetric	TEST-006
BR-006	PnL/ Sharpe/ Inventory	TR-005	metrics/ pnl.py	AgentMetric	TEST-006
BR-007	Game-Theoretic Analytics	TR-006	gametheory/ nash_solver.py,	EquilibriumAnalysis	TEST-008

Business/ Research	Product Feature	Technical	Component	Data	Test
			gametheory /best_response.py		
BR-008	Dashboard	TR-007	dashboard/ app.py, dashboard/ charts.py	(reads all)	TEST-010
BR-009	Reproducibility	TR-008	env seeding	SimulationRun	TEST-007
BR-010	Zero-Sum/ Spread Validation	TR-009	engine/ validators .py	Order, Trade	TEST-004, TEST-005

Interview Preparation

Product/Research Questions

1. What real-world assumption does traditional backtesting make that this project removes?
2. Who benefits from this kind of simulation, and how?
3. What is the MVP, and why were game-theory real-time updates and the C++ port excluded from it?
4. Why validate with non-learning baseline agents before trusting any RL agent?
5. What does “success” look like for this project — what would you show a skeptical evaluator?
6. What is explicitly out of scope, and why?
7. What happens if the matching engine has a subtle bug — how would you even notice?
8. What would you build next if you had another 12 weeks?
9. What’s the single biggest risk to this project succeeding?

Technical Questions

1. Why PettingZoo instead of a custom multi-agent interface?
2. Why priority queues for the order book, and what’s their complexity?
3. Why Ray RLlib instead of writing your own PPO/Q-learning loop?
4. Why Nashpy instead of implementing Lemke-Howson yourselves?
5. How do you guarantee the matching engine never crosses the spread?
6. How do you validate zero-sum inventory conservation?
7. How do you make a simulation run reproducible?
8. How would you scale the matching engine if throughput became a bottleneck?
9. How do you distinguish “the agent is still learning” from “the agent is stuck oscillating”?

10. How is the Bellman equation used in this project specifically?
11. When would the C++ port actually be justified?
12. How would you add a new agent archetype without touching the matching engine?

Architecture Questions

1. Why keep game-theoretic analysis post-hoc rather than real-time?
2. How would you isolate the matching engine from the RL training code if you later swap RLlib for something else?
3. How would you support many more concurrent agents?
4. How would you detect and diagnose MARL non-stationarity in practice?
5. How would you validate the Nashpy solver's output before trusting it on real logged data?
6. How would you extend this to calibrate against live (not just historical LOBSTER) order-flow data?

Scenario Questions

1. Your RL agent's reward curve is oscillating, not converging — what do you check first?
2. Two orders would cross the spread if matched incorrectly — how did your engine prevent that, and how would you prove it did?
3. A re-run with the same seed produces a different trade sequence — what's your debugging process?
4. GPU becomes unavailable mid-training — what's the fallback?
5. The Nashpy solver returns no equilibrium for a payoff matrix you expected one from — what do you investigate?
6. A mentor asks "how do you know your simulated market isn't just noise?" — what evidence do you show them?
7. Training throughput is too slow to finish in the time budget — what do you try before reaching for the C++ port?

Strong Answer Principle

Explain not just *what* was built, but *why* each architectural decision (Python-first engine, post-hoc game theory, RLlib over a custom loop) was made and what trade-off was consciously accepted.

Viva / Project Defense

Beginner

1. What is a limit order book?
2. What does "price-time priority" mean?
3. What is a bid and an ask?
4. What is the bid-ask spread?

5. What is reinforcement learning, briefly?
6. What is a reward in this project?
7. What is PnL?
8. What is a Nash equilibrium, in plain terms?
9. What is a Zero-Intelligence agent?
10. What is a Market-Maker agent?
11. Why use Python for this project?
12. What is PettingZoo?
13. What is a priority queue, and why use one here?
14. What is a random seed, and why does it matter here?
15. What is Sharpe ratio measuring?

Intermediate

1. Why does multi-agent RL make training harder than single-agent RL?
2. How does the Bellman equation drive the Q-learning update?
3. Why validate zero-sum inventory conservation every step, not just in tests?
4. How do you detect a spread-crossing bug?
5. Why start training with baseline (non-learning) agents first?
6. What is the Lemke-Howson algorithm computing?
7. Why is game-theoretic analysis done post-hoc instead of live?
8. How would you know if your RL agent actually “learned” something versus got lucky?
9. What are “stylized facts” of real markets, and why look for them in simulated data?
10. Why might GPU not be strictly necessary for this project?
11. What’s the difference between a limit order and a market order in your engine?
12. How do you handle a partial fill?
13. What’s the purpose of an ADR in this project’s documentation?
14. What’s the difference between HLD and LLD here?
15. Why keep the game-theory module separate from the matching-engine module?

Advanced

1. How would you formally prove your matching engine is correct, beyond test cases?
2. How would you extend the Nashpy-based analysis to more than two competing agent types?
3. How would you detect regime shifts or instability (flash-crash-like behavior) in the simulated market?
4. How would you calibrate your Zero-Intelligence “background noise” agents against the LOBSTER dataset rigorously?
5. How would you scale to many more concurrently-learning agents without the non-stationarity problem becoming unmanageable?
6. How would you design the promised C++ port so it’s a drop-in replacement behind the same PettingZoo interface?

7. How would you implement the Shapley-value attribution stretch goal to explain which agent behaviors drive volatility?
 8. How would you design an adversarial macro-shock injection mechanism and measure “recovery”?
 9. How would you distinguish a genuine Nash-equilibrium convergence from an artifact of your logging/sampling method?
 10. If your evaluator asked “is this economically realistic, or just a toy?”, how would you defend the model’s assumptions?
-

Final Project Quality Score

Score the implemented project after demonstration.

Dimension	Score
Problem Clarity	/10
Research/Product Value	/10
Engine Correctness	/10
Technical Complexity	/10
Architecture	/10
Code Quality	/10
RL Training Quality	/10
Game-Theoretic Rigor	/10
Testing	/10
Reproducibility	/10
Dashboard/UX	/10
Innovation (stretch goals attempted)	/10
Documentation	/10
Viva Defensibility	/10

Evaluation Guidance

1-3

Basic prototype; major gaps in engine correctness or RL training.

4-6

Functional project with baseline agents working and some RL training evidence.

7-8

Strong research-quality project: correct engine, demonstrable RL learning, credible game-theoretic analysis.

9-10

Exceptional depth: rigorous engine validation, clearly converging RL agents, defensible Nash-equilibrium findings, and at least one stretch goal meaningfully attempted.

Final Review

- Total Score:
- Strengths:
- Weaknesses:
- Critical Gaps:
- Recommended Improvements:

Distinction-Level Criteria

A distinction-level project should demonstrate:

- A provably correct matching engine (zero-sum + no-spread-crossing, backed by tests).
- RL agents with a clear, reported convergence story versus baselines.
- Credible, validated game-theoretic equilibrium analysis.
- Full reproducibility from seed + config.
- Clear architectural trade-offs the team can defend at viva.
- At least meaningful progress on one stretch goal (C++ port, adversarial shocks, or Shapley attribution).